

Aura Migration Document

Purpose: This document is the step-by-step specification a code-generation agent (in VS Code) will follow to rebuild the existing "Aura" jewellery recommendation app in a modern stack, while adding registration, login, session handling, route protection, and real images. This document contains **no application code** — only architecture, data models, flows, and an ordered execution plan.

Source app analyzed: `-jewellery-age-advisor/index.html`, `styles.css`, `data.js`, `app.js` (current repository state).

Target stack: Next.js 14+ (App Router), TypeScript, Tailwind CSS, shadcn/ui, Supabase (Auth + Postgres). Images stored locally under `/public/images`, served via `next/image`.

1. Feature Inventory (from current code)

1.1 Screens (2, client-side toggle via `.screen.active` class — no routing)

Screen 1 — Input form (`#form-screen`)

- `age`: `<input type="range">`, min 1, max 80, default 28. Live label update on `input event (#age-value)`.
- `relationship`: `<select>` — options: `self ("Myself")`, `daughter`, `mother`, `wife` (default selected), `friend`, `sister`.
- `occasion`: `<select>` — options: `birthday`, `wedding`, `anniversary` (default selected), `festival`, `everyday`, `graduation`.
- `budget`: `<input type="range">`, min 1000, max 200000, step 500, default 40000. Live label formatted as INR (`₹40,000`) via `formatINR()`.
- `style`: `<select>`, optional — options: `""` ("No preference"), `minimal`, `traditional`, `statement`, `modern`.
- Submit button: "Curate My Picks" — form `submit` handler calls `e.preventDefault()`, builds a `prefs` object, renders results, and switches to the results screen.

- No client-side validation beyond native HTML constraints (range min/max enforced by the browser; selects always have a valid value since none are empty-required).

Screen 2 — Results (#results-screen)

- Header: "← Refine details" back button (#back-btn) returns to form screen (values preserved, since the same DOM inputs are reused).
- Title: "Curated for {relationship label}" (e.g., "Curated for your wife").
- Subtitle: "Age {age} · {Occasion capitalized} · Up to {formatted budget}".
- **Re-filter bar** (no full re-submit needed):
 - refilter-budget slider — same 1000–200000/step 500 range, initialized to the submitted budget; input event updates currentPrefs.budget , re-renders subtitle and cards live.
 - refilter-style select — same options as the form's style select; change event updates currentPrefs.style and re-renders cards.
- **Results grid** (#results-grid): up to 6 cards, each with:
 - Placeholder image block (.card-image) — CSS gradient keyed by item.category (no real image).
 - Category label (uppercase, small).
 - Item name (serif heading).
 - Price (formatted INR).
 - "Why" explanation — one line of generated text (see §1.3).
- **No-results state**: shown when the filtered pool is empty ("No pieces match this budget yet — try widening the range.").

1.2 Dataset (data.js)

40 static items, each shaped as:

```
{ id: number, name: string, category: string, price: number,
  ageMin: number, ageMax: number, style: string, tags: string[] }
```

- category values observed: Ring , Necklace , Earrings , Bracelet , Bangle , Anklet , Pendant , Set , Chain .
- style values observed: minimal , traditional , statement , modern .
- Age bands used by the dataset (matching the spec's brackets): 1–12, 13–19, 20–29, 30–45, 46–60, 61–80, plus four **occasion-anchored items** with wider/

overlapping age ranges (ids 37–40: bridal heirloom set 18–80, anniversary pendant 18–80, festival jhumka set 13–80, graduation bracelet 15–30) — these exist specifically so wedding/anniversary/festival/graduation occasions always have a plausible match regardless of the submitted age.

- `tags` are free-form strings used purely for scoring (e.g., `"heirloom"`, `"wedding"`, `"low-maintenance"`) — no separate taxonomy/table backs them today.

1.3 Recommendation / scoring logic (`app.js`)

Pure function pipeline, deterministic, no randomness:

1. **`ageBracket(age)`** — maps age to one of 6 brackets (`kids` ≤ 12 , `teen` ≤ 19 , `twenties` ≤ 29 , `midlife` ≤ 45 , `mature` ≤ 60 , `senior` > 60), each carrying a `tone` string used in the "why" text fallback.
2. **`heirloomSkew(relationship, occasion)`** — returns `true` when `occasion` is `wedding` or `anniversary` **and** `relationship` is `wife` or `mother`. This is the explicit override called out in the spec ("`wedding + wife`" skews heirloom regardless of age).
3. **`occasionTagBoost(occasion)`** — a fixed map from `occasion` $\rightarrow$ array of tags that should score higher (e.g., `wedding` $\rightarrow$ [`"wedding"`, `"heirloom"`, `"bridal"`, `"statement"`]).
4. **`scoreItem(item, prefs)`** — additive score:
 5. `+10` if `age` falls within [`ageMin`, `ageMax`]; otherwise a decaying partial score (`max(0, 6 - distance*0.5)`) so near-miss ages aren't zeroed out.
 6. `+6` if `heirloomSkew` is true and the item has an `"heirloom"` tag or `style` is `traditional`/`statement`.
 7. `+3` per matching tag from `occasionTagBoost(occasion)`.
 8. `+5` if `prefs.style` is set and matches `item.style` exactly.
 9. Budget: `+4` plus `+(price/budget)*2` if `price` $\leq$ `budget` (rewards using more of the budget); **`-8 if over budget`** (over-budget items are excluded from the pool entirely before scoring even runs, so this penalty is currently dead code / defensive only).
10. **`getRecommendations(prefs, allowOverBudget)`** — filters dataset to `price` $\leq$ `budget` (unless `allowOverBudget`), scores remaining items, sorts descending, returns top 6. Note: `allowOverBudget` is passed as `false` everywhere in the current UI — there is no "show me over-budget options anyway" affordance today.

11. `whyText(item, prefs)` — generates the one-line explanation: "Picked for {relationship phrase} ({age}), {occasion}: {reason}." where `reason` is either the heirloom-skew sentence or the age bracket's `tone` string. This is a template-string generator, not free-text AI generation — fully rule-based.

1.4 State management

- Single `currentPrefs` object held in a closure in `app.js`, mutated in place by the re-filter controls. No persistence (refreshing the page resets to the form screen with default slider values); no URL state, no `localStorage`.

1.5 Formatting helpers

- `formatINR(n) → "₹" + n.toLocaleString("en-IN")`.
- `capitalize(s) →` simple first-letter uppercase, used for the occasion label in the subtitle.
- `relationshipLabel(r) / relationshipTone(r)` — two separate maps (title-case vs. sentence-fit phrasing) for rendering the relationship in different contexts (results title vs. why-text).

2. Target Architecture (Next.js App Router)

Note: every path below is a proposed file in the not-yet-created Next.js project — none of these exist in the current repository. Only §1 and the "Referenced Paths" section at the end describe files verified against the actual `-jewellery-age-advisor` repo today.

```
aura/
├─ app/
│   ├─ layout.tsx           # Root layout: fonts, AuthHeader, global providers
│   ├─ page.tsx             # Home = recommendation form (Screen 1 equivalent)
│   ├─ globals.css          # Tailwind base + design tokens (see §7)
│   ├─ results/
│   │   └─ page.tsx         # Results screen; reads prefs from search params/state
│   ├─ login/
│   │   └─ page.tsx         # Login page (public)
│   ├─ register/
│   │   └─ page.tsx         # Registration page (public)
│   ├─ favorites/
│   │   └─ page.tsx         # NEW: saved/favorited items (protected – see §4)
│   ├─ auth/
│   │   └─ callback/route.ts # Supabase OAuth/email-confirmation callback handler
```

```

|   └─ api/
|       └─ favorites/route.ts          # Server route: add/remove a favorite (protected)
├─ components/
|   └─ auth/
|       ├── RegistrationForm.tsx
|       ├── LoginForm.tsx
|       └─ AuthHeader.tsx
|   └─ recommendations/
|       ├── RecommendationForm.tsx    # Screen 1 form, shadcn Slider/Select
|       ├── ResultsGrid.tsx          # Grid + no-results state + re-filter bar
|       └─ RefilterBar.tsx           # Budget slider + style select (results-screen-only
controls)
|   └─ JewelleryCard.tsx             # Single card, next/image, why-text, favorite button
├─ ui/                               # shadcn/ui generated primitives (Button, Slider, Select,
Card, Input, Label, Alert...)
├─ lib/
|   ├── supabase/
|   │   ├── client.ts                # Browser Supabase client (createBrowserClient)
|   │   ├── server.ts               # Server Supabase client (createServerClient, cookies)
|   │   └─ middleware.ts             # Helper used by middleware.ts to refresh session
|   └─ recommendation-engine.ts      # Ported ageBracket/heirloomSkew/scoreItem/
getRecommendations/whyText
|   └─ format.ts                     # formatINR, capitalize, relationshipLabel/Tone
├─ types/
|   ├── jewellery.ts                 # JewelleryItem, Prefs
|   └─ auth.ts                       # AppUser, FavoriteRecord
├─ data/
|   └─ jewellery.ts                  # Ported 40-item dataset (static, typed) – see §3
├─ public/
|   └─ images/
|       └─ jewellery/                # Real product photos – see §6
├─ middleware.ts                     # Route protection (see §4)
├─ tailwind.config.ts
├─ components.json                   # shadcn/ui config
└─ .env.local                        # NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY,
SUPABASE_SERVICE_ROLE_KEY (server-only)

```

Key structural decisions:

- The original single-page toggle (`.screen.active`) becomes **two real routes** (`/` and `/results`) so results are linkable/shareable and back/forward navigation works natively — a behavior improvement over the original, not a regression.
- Preferences travel from `/` to `/results` via **URL search params** (`?age=28&relationship=wife&occasion=anniversary&budget=40000&style=`) rather than component state, so the results page is directly shareable/bookmarkable and survives a refresh (the original app could not survive a refresh at all).
- The dataset stays a static TypeScript module (`data/jewellery.ts`), not a Supabase table — per your "no backend for recommendations" original

constraint, only auth + favorites need a database. (Flagged as an open question in §9 in case you'd rather move the dataset into Postgres too.)

3. Data Modeling

3.1 TypeScript types (types/jewellery.ts)

```
export type JewelleryCategory =
  | "Ring" | "Necklace" | "Earrings" | "Bracelet"
  | "Bangle" | "Anklet" | "Pendant" | "Set" | "Chain";

export type JewelleryStyle = "minimal" | "traditional" | "statement" | "modern";

export interface JewelleryItem {
  id: number;
  name: string;
  category: JewelleryCategory;
  price: number;           // INR, integer
  ageMin: number;
  ageMax: number;
  style: JewelleryStyle;
  tags: string[];
  imagePath: string;       // NEW – e.g. "/images/jewellery/ring-cocktail-statement.jpg"
  imageAlt: string;        // NEW – accessibility text, derived from `name`
}

export type Relationship = "self" | "daughter" | "mother" | "wife" | "friend" | "sister";
export type Occasion = "birthday" | "wedding" | "anniversary" | "festival" | "everyday" |
  "graduation";

export interface RecommendationPrefs {
  age: number;
  relationship: Relationship;
  occasion: Occasion;
  budget: number;
  style: JewelleryStyle | "";
}
```

3.2 Auth/user types (types/auth.ts)

```
export interface AppUser {
  id: string;           // Supabase auth.users.id (uuid)
  email: string;
  createdAt: string;
}

export interface FavoriteRecord {
```

```
id: string;           // uuid, primary key
userId: string;       // FK -> auth.users.id
jewelleryItemId: number; // FK into the static dataset by `id` (see note below)
createdAt: string;
}
```

Note: because the jewellery dataset stays a static file (not a DB table), `favorites.jewellery_item_id` is stored as a plain integer referencing `JewelleryItem.id` from the static dataset — there is no DB foreign key constraint enforcing referential integrity against a table that doesn't exist. This is flagged again in §9.

3.3 Supabase schema (Postgres)

Supabase's built-in `auth.users` table covers registration/login (email + hashed password) — **no custom users table is required** unless you want to store additional profile fields. Recommended minimal schema:

```
-- Optional profile extension (only if you need fields beyond auth.users)
create table public.profiles (
  id uuid references auth.users(id) on delete cascade primary key,
  email text not null,
  created_at timestamptz not null default now()
);

-- Favorites (the one new persisted entity this migration introduces)
create table public.favorites (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  jewellery_item_id integer not null, -- references the static dataset's `id`, not a DB FK
  created_at timestamptz not null default now(),
  unique (user_id, jewellery_item_id)
);

alter table public.favorites enable row level security;

create policy "Users can view their own favorites"
  on public.favorites for select
  using (auth.uid() = user_id);

create policy "Users can insert their own favorites"
  on public.favorites for insert
  with check (auth.uid() = user_id);

create policy "Users can delete their own favorites"
```

```
on public.favorites for delete
using (auth.uid() = user_id);
```

- Row Level Security (RLS) is mandatory here since Supabase's anon key is exposed client-side — without RLS any signed-in user could read/write any other user's favorites.
- No `jewellery_items` table is created in this pass (dataset stays static per §2) — if that changes, add a `jewellery_items` table mirroring `JewelleryItem` and make `favorites.jewellery_item_id` a real foreign key.

4. Auth Design

4.1 Supabase client wiring

- **Browser client** (`lib/supabase/client.ts`): `createBrowserClient` from `@supabase/ssr`, used inside Client Components (`RegistrationForm`, `LoginForm`, `AuthHeader`, `JewelleryCard`'s favorite button).
- **Server client** (`lib/supabase/server.ts`): `createServerClient` from `@supabase/ssr`, reading/writing cookies via `next/headers` — used in Server Components and Route Handlers (`app/api/favorites/route.ts`) to read the current session and enforce RLS-backed queries server-side.
- **Middleware** (`middleware.ts`, root-level): uses the `@supabase/ssr` middleware helper to refresh the auth session cookie on every request (Supabase's access token is short-lived; this keeps it alive without the user noticing) and to perform route protection (§4.4).

4.2 Sign-up flow

1. `RegistrationForm` (Client Component) collects email + password + confirm-password.
2. Client-side validation before submit:
 3. Email: matches a standard email regex / HTML5 `type="email"` constraint.
 4. Password: minimum 8 characters, at least one letter and one number (mirrors "password length/strength" requirement — exact policy is a §9 open question).
 5. Confirm-password must match password.
6. On submit: `supabase.auth.signUp({ email, password })`.

7. Two sub-cases depending on your Supabase project's email confirmation setting (this is a project-level toggle, not code):
 8. **Email confirmation required** (Supabase default): show a "check your email to confirm" message; user is not signed in yet. `app/auth/callback/route.ts` handles the confirmation link redirect and exchanges the code for a session.
 9. **Email confirmation disabled**: user is signed in immediately after `signUp`; redirect to `/` (or `/favorites`, see §9).
10. Errors surfaced inline: duplicate email ("An account with this email already exists"), weak password (from Supabase's own policy if configured), network error.

4.3 Sign-in flow

1. `LoginForm` (Client Component) collects email + password.
2. On submit: `supabase.auth.signInWithPassword({ email, password })`.
3. Error handling: Supabase returns a generic "Invalid login credentials" for both wrong-password and unknown-email cases (this is intentional on Supabase's part, to avoid leaking which emails are registered) — surface that message as-is rather than trying to distinguish the two cases.
4. On success: redirect to the page the user was trying to reach before being redirected to `/login` (see §4.4), defaulting to `/`.

4.4 Session persistence & route protection

- Supabase Auth persists the session via HTTP-only cookies (through `@supabase/ssr`), so sessions **survive page refreshes and new tabs** automatically — no custom `localStorage`/token handling needed.
- `middleware.ts` runs on every request matching a config matcher (excluding static assets), refreshes the session, and redirects unauthenticated users away from protected routes to `/login?redirectedFrom=<path>`.
- **Protected routes (this migration's default assumption, per your instructions):**
 - `/favorites` — viewing and managing saved items requires login.
 - `POST /api/favorites` and `DELETE /api/favorites` — require login (also enforced server-side by RLS as defense-in-depth, not just middleware).
- **Explicitly NOT protected (open/public):**

- `/` (recommendation form) and `/results` (results grid) — browsing and getting recommendations requires no account, matching the original app's fully-open behavior.
- `/login`, `/register` — public by definition; middleware should redirect an **already-authenticated** user away from these two back to `/`.

4.5 Logout

- `AuthHeader` (rendered in `app/layout.tsx`, present on every page) shows either "Sign in / Register" links (logged out) or the user's email + a "Log out" button (logged in), sourced from a server-rendered session check passed down, hydrated client-side for the button's `onClick`.
- Logout calls `supabase.auth.signOut()` (browser client) then router-refreshes/redirects to `/`.

5. Component Breakdown

Component	Type	Responsibility
<code>RegistrationForm</code>	Client	Email/password/confirm fields (<code>shadcn Input</code> , <code>Label</code> , <code>Button</code>), validation, calls <code>supabase.auth.signUp</code> , inline error/success states via <code>shadcn Alert</code> .
<code>LoginForm</code>	Client	Email/password fields, calls <code>supabase.auth.signInWithPassword</code> , inline error via <code>shadcn Alert</code> , "Forgot password?" link (scope flagged in §9).
<code>AuthHeader</code>	Server (with a small Client sub-part for the logout button)	Reads session server-side; renders brand mark + either auth links or "{email} · Log out"; replaces/extends the original static <code>.topbar</code> .
<code>RecommendationForm</code>	Client	Direct port of Screen 1: age slider (<code>shadcn Slider</code>), relationship/occasion/style selects (<code>shadcn Select</code>), budget slider, submit → navigates to <code>/results?...query params...</code> .
<code>ResultsGrid</code>	Server (fetches/filters) + Client island for re-filter	Reads prefs from <code>searchParams</code> , calls <code>getRecommendations</code> (ported logic), renders <code>JewelleryCard</code> list or the no-results state; hosts <code>RefilterBar</code> .

Component	Type	Responsibility
RefilterBar	Client	Budget slider + style select scoped to the results page; on change, updates the URL search params (via <code>useRouter().replace()</code>) so the grid re-renders — this preserves the original's "live re-filter without re-entering everything" behavior.
JewelleryCard	Client (favorite button needs auth state) or Server + Client sub-component	Renders <code>next/image</code> (replacing the gradient div), category, name, price, why-text, and a heart/favorite toggle button that: if logged out, redirects to <code>/login</code> ; if logged in, calls the favorites API route.
ui/* (shadcn)	—	Generated primitives: <code>Button</code> , <code>Input</code> , <code>Label</code> , <code>Select</code> , <code>Slider</code> , <code>Card</code> , <code>Alert</code> , <code>Separator</code> . Installed via the shadcn CLI, not hand-written.

6. Image Sourcing Plan

- **Source:** royalty-free stock photography from **Unsplash** or **Pexels** (both allow free commercial use without attribution, appropriate for this MVP). Search terms per category, e.g. "gold cocktail ring", "pearl necklace set", "gold bangles", "kids gold anklet", "temple jewellery earrings", "diamond tennis bracelet".
- **Folder:** `public/images/jewellery/`.
- **Naming convention:** kebab-case, `{category-lowercase}-{descriptive-slug}.jpg`, matching the item's `name` closely enough to be traceable, e.g.:
 - id 1 "Tiny Gold Studs" → `earrings-tiny-gold-studs.jpg`
 - id 19 "Solitaire Diamond Ring" → `ring-solitaire-diamond.jpg`
 - id 25 "Temple Jewellery Set" → `set-temple-jewellery.jpg`
- **Mapping:** each of the 40 dataset entries in `data/jewellery.ts` gets an explicit `imagePath: "/images/jewellery/<file>.jpg"` field — no runtime guessing/pattern-matching from `name`, so a missing file is a build-time-visible bug rather than a silent broken `<img>`.
- **Fallback:** keep the original CSS-gradient-by-category as the `next/image` `placeholder="blur"` `blurDataURL` substitute or as an `onError` fallback, so a missing/broken image degrades gracefully to the current visual style instead of a broken-image icon.
- **Sizing:** source images at $\geq 800 \times 800$ px (square crop preferred) so `next/image` can generate responsive sizes down to the card's $\sim 240 \times 160$ display size without upscaling artifacts.

- **Licensing note:** even though Unsplash/Pexels licenses don't require attribution, keeping a simple `public/images/jewellery/CREDITS.md` with source URLs per image is good practice and trivial to generate alongside the images.

7. Styling Migration (CSS → Tailwind + shadcn/ui)

7.1 Design tokens (`tailwind.config.ts` theme extension)

Current CSS variable	Value	Tailwind token
<code>--cream</code>	<code>#FBF6EE</code>	<code>colors.cream.DEFAULT</code>
<code>--ivory</code>	<code>#FFFDF8</code>	<code>colors.ivory.DEFAULT</code>
<code>--gold</code>	<code>#C6952C</code>	<code>colors.gold.DEFAULT</code>
<code>--gold-light</code>	<code>#E8C874</code>	<code>colors.gold.light</code>
<code>--rose-gold</code>	<code>#B76E79</code>	<code>colors.rose.DEFAULT</code>
<code>--rose-gold-light</code>	<code>#E8B4BC</code>	<code>colors.rose.light</code>
<code>--ink</code>	<code>#2B2420</code>	<code>colors.ink.DEFAULT</code>
<code>--ink-soft</code>	<code>#6B5F53</code>	<code>colors.ink.soft</code>
<code>--border-soft</code>	<code>#E9DFCB</code>	<code>colors.border.soft</code> (or shadcn's <code>--border</code> CSS var override)
<code>--radius: 18px</code>	—	<code>borderRadius.xl</code> custom value, also feeds shadcn's <code>--radius</code> CSS var so its primitives inherit the same rounding.
<code>--shadow</code>	<code>0 10px 30px rgba(43,36,32,0.08)</code>	<code>boxShadow.soft</code> custom key.

Fonts: Playfair Display (headings) and Poppins (body) loaded via `next/font/google` in `app/layout.tsx` and mapped to `fontFamily.serif` / `fontFamily.sans` in the Tailwind config — replacing the current `<link>`-tag Google Fonts loading.



7.2 Component-level mapping

Original CSS class	Tailwind/shadcn equivalent
<code>.advisor-card</code>	shadcn Card + <code>className="max-w-[560px] mx-auto p-9 bg-ivory border-border-soft shadow-soft rounded-xl"</code>
<code>.cta-button</code> (gold→rose-gold gradient pill)	shadcn Button with a custom <code>variant="gradient"</code> added to <code>buttonVariants</code> — <code>bg-gradient-to-r from-gold to-rose-gold text-white rounded-full</code>
<code>input[type=range]</code> custom thumb styling	shadcn Slider (Radix-based) restyled via its own <code>className</code> /CSS vars to use the gold thumb + gold/rose-gold track gradient
<code>.card</code> / <code>.card-image</code> / <code>.card-body</code>	shadcn Card, CardHeader (holds the <code>next/image</code>), CardContent (category/name/price/why)
<code>.results-header</code> / <code>.link-btn</code>	Plain Tailwind utility classes + shadcn Button <code>variant="link"</code> for the back button
<code>.refilter-bar</code>	shadcn Card styled as a filter toolbar, flex-wrap layout preserved via Tailwind <code>flex flex-wrap items-center gap-4</code>
<code>.screen</code> / <code>.screen.active</code> fade-in	Replaced entirely by real route transitions; a subtle Tailwind <code>animate-in fade-in</code> (via <code>tailwindcss-animate</code> , which shadcn installs by default) on page mount replicates the original fade.
<code>@media (max-width: 560px)</code> overrides	Tailwind responsive prefixes (<code>sm:</code> , default mobile-first) applied directly on each component instead of a separate media-query block.

7.3 New screens (login/register) styling

- Reuse the `.advisor-card` → shadcn Card treatment for `LoginForm/RegistrationForm` so they visually match the original form's card (same max-width, padding, shadow, radius).
- AuthHeader's "logged-in" state (email + logout) styled as small `text-ink-soft` text next to a Button `variant="ghost" size="sm"` for logout, sitting in the existing `.topbar` region — no new visual language introduced, just an additional row of content in the header.

8. Step-by-Step Migration Plan

1. **Scaffold project** — `npx create-next-app@latest aura --typescript --tailwind --app`; verify dev server boots.

2. **Set up Supabase project & schema** — create a Supabase project; run the SQL in §3.3 (profiles optional, favorites + RLS policies mandatory); capture `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` into `.env.local`; decide the email-confirmation toggle now (feeds §4.2 and §9).
3. **Add Tailwind theme + shadcn/ui** — extend `tailwind.config.ts` with the tokens in §7.1; run the shadcn `init` and add commands for `button`, `input`, `label`, `select`, `slider`, `card`, `alert`, `separator`; load `next/font/google` for Playfair Display + Poppins in `app/layout.tsx`.
4. **Port types & data with images** — write `types/jewellery.ts`, `types/auth.ts`; transcribe the 40-item dataset into `data/jewellery.ts` with the new `imagePath` / `imageAlt` fields; source and place images per §6; build `public/images/jewellery/CREDITS.md`.
5. **Build auth pages & middleware** — implement `lib/supabase/{client,server,middleware}.ts`; build `RegistrationForm`, `LoginForm`, `AuthHeader`; wire `app/login/page.tsx`, `app/register/page.tsx`, `app/auth/callback/route.ts`; implement `middleware.ts` route protection per §4.4; manually verify: sign-up → (confirm if required) → sign-in → refresh page (still logged in) → logout.
6. **Port recommendation components & logic** — translate `ageBracket` / `heirloomSkew` / `occasionTagBoost` / `scoreItem` / `getRecommendations` / `whyText` / `formatINR` / `capitalize` / `relationshipLabel` / `relationshipTone` into `lib/recommendation-engine.ts` and `lib/format.ts` (types added, logic unchanged); build `RecommendationForm`, `ResultsGrid`, `RefilterBar`, `JewelleryCard`; wire `app/page.tsx` and `app/results/page.tsx` to pass prefs via search params.
7. **Add favorites (new feature, protected)** — `app/api/favorites/route.ts` (GET list / POST add / DELETE remove, using the server Supabase client + RLS); favorite toggle button in `JewelleryCard` (redirects to `/login` if logged out); `app/favorites/page.tsx` protected results-style grid reading the user's saved items.
8. **Verify against original screenshots** — side-by-side comparison of `/` vs. the original form screen and `/results` vs. the original results screen for: default slider values, INR formatting, card layout/spacing, gradient/color fidelity, the no-results message, and the wedding+wife heirloom-skew behavior producing the same top picks as the original for a fixed test input.
9. **Deploy** — push to GitHub, import into Vercel, set the three Supabase env vars in Vercel's project settings (marking the service-role key, if used server-side only, as a server-only/non-public env var), confirm the Supabase project's allowed redirect URLs include the deployed domain for the auth callback.



9. Risks / Gaps — Flagged for Your Review Before Code Generation

1. **Password policy specifics** — the prompt says "basic validation (valid email, password length/strength)" without an exact policy. This document assumes **min 8 chars + at least 1 letter + 1 number**; confirm or adjust before generation (Supabase also has its own configurable minimum, which should match).
2. **Password reset flow — out of scope or in?** Not requested explicitly; this document does **not** include a "Forgot password" flow in the step plan, only a placeholder link in `LoginForm`. Confirm whether it should be built now or deferred.
3. **Logged-out user's results — what happens to them?** Per your instruction, browsing/results stay open to everyone; only favoriting requires login. Confirm this reading is correct — an alternative reading of "route protection" could have meant gating the whole `/results` page instead.
4. **Email confirmation on/off RESOLVED (Epic 2, Story 2.3): confirmation required — see below.** — this is a Supabase project setting, not code, and changes the post-signup UX (immediate session vs. "check your email"). Needs a decision in Step 2 of §8 before `RegistrationForm`'s success-state copy can be finalized.
5. **Dataset location (static file vs. Supabase table)** — this document keeps the 40-item jewellery dataset as a static TypeScript file (matching "no backend" for recommendations), with `favorites.jewellery_item_id` as a plain integer with **no real foreign key** back to it. If you'd rather have an admin-editable catalog later, the dataset should move into a `jewellery_items` Postgres table now instead — flag before Step 4 of §8 if so, since it changes §3 and §6 significantly.
6. **Image copyright/consistency** — Unsplash/Pexels sourcing is manual curation, not automated; expect visual style to vary slightly across 40 hand-picked photos. If a single consistent product-photography look is required, that likely means commissioning/generating a uniform image set instead — flag if this matters for launch quality. **UPDATE (Epic 4):** real curation wasn't feasible in the code-gen session; all 40 images are deterministic `picsum.photos` placeholders for now (see `public/images/jewellery/CREDITS.md`) — replace before launch, same filenames/naming convention so no dataset changes are needed later.
7. **Epic 4 data-model correction RESOLVED (Epic 6):** ported `ageBracket / heirloomSkew / occasionTagBoost / scoreItem / getRecommendations / whyText` per this document's §1.3 design (not the real `app.js`'s `scoreItem / recommend`),

consistent with Epic 4's schema choice. `occasionTagBoost`'s per-occasion tag lists and the `relationshipLabel/relationshipTone` maps were underspecified here (only one example given) — filled in with the minimal literal extrapolation in `lib/recommendation-engine.ts/lib/format.ts`, flagged there for review. A fixed regression case (age 34/wife/wedding/₹200,000/no style) is locked in `lib/recommendation-engine.test.ts` and verified live against the running app.

Original text: this document's §1.2/§1.3 (and the `tags/scoring-function` names used throughout §3.1 and §8 Step 6) were written from an earlier analysis that no longer matches the actual `data.js/app.js` in the repo. The real dataset uses separate `occasionTags/relationshipTags` arrays (not one generic `tags` field) and a `gradient` string per item (not a category-keyed gradient map); the real scoring engine is `scoreItem/recommend` in `app.js`, not `heirloomSkew/occasionTagBoost/whyText`. Per an explicit decision, Epic 4 kept this document's schema (`tags: string[]`, computed as the union of the real `occasionTags + relationshipTags`) rather than reshaping to match the live code exactly — flagging here since **Epic 6's logic port must resolve the same conflict** (either port the doc's described `heirloomSkew/whyText` design, or port the real `scoreItem/recommend` code) before proceeding. Also: the real dataset includes a "Hair Jewellery" category (item 37) absent from this doc's category list in §3.1 — added to `types/jewellery.ts`'s `JewelleryCategory` union rather than mis-tagging that item.

1. **allowOverBudget dead parameter** — the original `getRecommendations(prefs, allowOverBudget)` accepts a flag that's never set to `true` anywhere in the current UI. This migration ports it as-is (harmless), but note it's an unused capability, not a bug to "fix" unless you want to expose it as a new "show over-budget options" affordance.
2. **Relationship-based favorites scoping** — favorites are stored per-user only, not per-relationship/occasion context (i.e., favoriting the same physical item while shopping for "wife/anniversary" vs. "sister/birthday" collapses to one row via the `unique(user_id, jewellery_item_id)` constraint). Confirm this is acceptable, or whether favorites should be scoped by the search context too.
3. **Rate limiting / abuse protection on auth routes** — not addressed in this document; Supabase has some built-in protections, but no additional app-level rate limiting is planned. Flag if this is a launch requirement.



Referenced Paths

The following files were analyzed to produce this migration document:

High Relevance

- `-jewellery-age-advisor/app.js` - Source of the entire recommendation/scoring engine (ageBracket, heirloomSkew, occasionTagBoost, scoreItem, getRecommendations, whyText) and all DOM/event wiring being ported to Next.js components.
- `-jewellery-age-advisor/data.js` - Source of the 40-item jewellery dataset and its schema, forming the basis for `types/jewellery.ts` and `data/jewellery.ts` in the target architecture.
- `-jewellery-age-advisor/index.html` - Source of the two-screen structure, form fields, and results-screen re-filter bar being mapped to `app/page.tsx` and `app/results/page.tsx`.

Medium Relevance

- `-jewellery-age-advisor/styles.css` - Source of the design tokens (colors, radius, shadow, fonts) and per-element styling being mapped to the Tailwind config and shadcn/ui components in §7.
- `-jewellery-age-advisor/README.md` - Documents original run/deploy assumptions and constraints (no backend, no build tooling) that this migration explicitly changes.